Comparing Two Machine Learning Tools for Predicting Injuries in New York City Car Crashes

Jay Cespedes
Big Data Machine Learning Course
Final Project
Miami, USA

***Abstract*—**This paper compares two popular machine learning tools — PySpark MLlib and Scikit-Learn — using real crash data from New York City. I used one million records to teach computer models how to predict whether a car crash caused an injury or not. I tested two types of models (Logistic Regression and Random Forest) in both tools after carefully cleaning the data and making sure the models could not "cheat." When I ran the experiments on Google's cloud computers, both tools were about equally good at telling the difference between crashes that caused injuries and crashes that did not. Their scores were very close (around 0.74 to 0.76). PySpark was better at handling the full large dataset, while Scikit-Learn was faster when I only used a smaller sample of the data. This paper explains the practical differences so that students and beginners can understand when to use each tool.

***Keywords*—**Big Data, Machine Learning, PySpark, Scikit-Learn, Car Crashes, Injury Prediction, Google Cloud

I. Introduction

Car crashes are a serious problem in cities. Being able to predict which crashes are more likely to cause injuries can help emergency services, city planners, and policy makers. New York City publishes a free public dataset that records every reported motor vehicle collision. This dataset contains millions of rows with information such as the time of the crash, the location, the type of vehicles involved, and what caused the crash.

Machine learning is a way for computers to learn patterns from data and make predictions. Two of the most common tools for machine learning are:

- Scikit-Learn — a popular Python library that works well when all the data fits on one computer.
- PySpark MLlib — a tool built for very large datasets that can use many computers working together at the same time.

In this project I ask three simple questions:

- How well do the two tools predict whether a crash caused an injury?
- Which tool is faster and easier to use with a large dataset?
- What steps are needed to clean the data so the models give honest results?

I answer these questions by building and comparing models using one million real crash records on a Google Cloud cluster.

II. The Dataset and How I Cleaned It

I started with the official New York City Motor Vehicle Collisions dataset. The raw data had problems that needed fixing before any machine learning could be done:

- Some location coordinates were clearly wrong (for example, latitude and longitude values of zero or numbers far outside New York City). I removed those rows.
- Many text fields had inconsistent capitalization or missing values. I standardized them and replaced missing values with the word "UNKNOWN."
- I created simple time-based features such as the hour of the day, the day of the week, the month, and the year.
- I created a clear yes/no target: has_injury = 1 if anyone was hurt or killed, and 0 if no one was injured.

After cleaning, I kept exactly 1,000,000 good records. About 25% of these crashes involved at least one injury. The cleaned data was stored on the cluster so both tools could use it.

III. How I Built the Models

A. Choosing the Right Information (Features)

One of the most important decisions was what information to give the models. I deliberately did NOT give the models the columns that already counted how many people were injured. If I had included those columns, the models would have been able to "cheat" by simply looking at the answer. This is called data leakage and it makes results look better than they really are.

Instead, the models only received information that would be known before knowing the injury outcome:

- Where the crash happened (borough, latitude, longitude)
- When the crash happened (hour, day of week, month, year)
- What caused the crash and what types of vehicles were involved

Text fields were turned into numbers using standard techniques (StringIndexer and OneHotEncoder). Numbers were scaled so that large and small values could be compared fairly.

B. The Two Types of Models I Tested

I tested the same two algorithms in both tools:

- Logistic Regression — a simple and fast model that draws a straight decision boundary.
- Random Forest — a more complex model made of many decision trees working together.

I used similar settings in both tools so the comparison would be fair.

IV. How I Ran the Experiments

All experiments were run on a Google Cloud Dataproc cluster (one master computer and two worker computers). PySpark MLlib was given the full 1,000,000 rows. Because Scikit-Learn runs on a single computer, I gave it a random sample of about 60,000 rows so it would not run out of memory. In both cases I used 80% of the data for training and 20% for testing.

I measured how good the models were using several common scores:

- AUC — how well the model ranks injury crashes higher than non-injury crashes (higher is better, 1.0 is perfect)
- Accuracy — the percentage of correct predictions
- F1-score, Precision, and Recall — scores that show how well the model handles the injury class
- Training time — how many seconds the model needed to learn

V. Results

Table I shows the final scores for all four combinations.

TABLE I

How Well Each Model Performed

Model	Tool	AUC	Accuracy	F1	Precision	Recall
Logistic Regression	PySpark	0.7617	0.7955	0.7575	0.7880	0.7955
Random Forest	PySpark	0.7388	0.7498	0.6426	0.8124	0.7498
Logistic Regression	Scikit-Learn	0.7586	0.7948	0.4231	0.7199	0.2996
Random Forest	Scikit-Learn	0.7490	0.7501	0.0112	0.8947	0.0056

The most important finding is that both tools produced very similar AUC scores (between 0.74 and 0.76). This means they were about equally good at ranking crashes by injury risk. Logistic Regression performed better than Random Forest in most measures. Scikit-Learn's F1 and Recall scores were much lower because it only saw a smaller sample of the data and because the default decision threshold does not handle imbalanced classes well. PySpark was able to train on the full one million rows, while Scikit-Learn needed a smaller sample to avoid running out of memory.

VI. What the Results Mean

The similar AUC scores tell us that the choice of tool does not dramatically change how well the model can rank crashes. The real differences appear in other areas:

- PySpark can use the entire dataset without sampling, which is important when you want the model to see every pattern in the data.
- Scikit-Learn is simpler to write and faster when the data is small enough to fit in one computer's memory.
- When the data is imbalanced (only 25% of crashes caused injuries), extra care is needed with Scikit-Learn to improve recall.

A limitation of this study is that I did not spend a long time tuning every possible setting of the models. I also used only one train/test split. Future work could try more advanced models or techniques that better handle the class imbalance.

VII. Conclusion

I built machine learning models that predict whether a New York City car crash caused an injury. After carefully cleaning one million records and preventing the models from cheating, I compared PySpark MLlib and Scikit-Learn. Both tools achieved similar ranking quality (AUC around 0.74–0.76). Logistic Regression was the stronger model overall. PySpark handled the full large dataset better, while Scikit-Learn was faster on a smaller sample. These results can help students and practitioners decide which tool to use when working with large public-safety datasets.

References

[1] City of New York, “Motor Vehicle Collisions – Crashes,” NYC Open Data. [Online]. Available: https://data.cityofnewyork.us/Public-Safety/Motor-Vehicle-Collisions-Crashes/h9gi-nx95

[2] Apache Software Foundation, “Apache Spark MLlib.” [Online]. Available: https://spark.apache.org/mllib/

[3] Google Cloud, “Dataproc documentation.” [Online]. Available: https://cloud.google.com/dataproc/docs